



ELECTRÓNICA DIGITAL III

CLASE 6 Módulo ADC

Ing. Emiliano Migliore

Módulo ADC

La LPC1769 cuenta con un solo bloque conversor analógico-digital de aproximaciones sucesivas **AD0CR**, el cual presenta una tasa de conversión de **12-bit** a una frecuencia máxima de muestreo $f_{m(max)}$ de 200KHz.

Cuenta con **8 canales** de entrada, correspondiente a los pines analógicos, y el valor digitalizado de 12-bit se almacena en los respectivos registros ADDR[0:7].

La frecuencia de muestreo del ADC se determina en base al criterio de Nyquist para las componentes de máxima frecuencia de las señales de entrada f_{max_i} y en base al número de canales utilizados n_{CH} como:

$$f_m = 2 \left(\sum_{i=0}^7 f_{max_i} \right) < \frac{f_{m(max)}}{n_{CH}}$$

Las **modalidades de disparo** del ADC, son:

- **Disparo Continuo o en Ráfaga (Burst Mode):** El ADC mantiene una conversión continua de los canales seleccionados, guardando los datos digitalizados en los respectivos registros, y sobrescribiéndolos en cada conversión.
 - ADC_START_CONTINUOUS

Las conversiones se realizan en 64 ciclos de reloj, por tanto, la frecuencia de trabajo del ADC está dada como:

$$f_{ADC} = 64f_{m(max)} = 12,8MHz > f_m$$

Módulo ADC

- **Disparo Controlado (No-Burst Mode):** El ADC mantiene una conversión de los canales seleccionados en un tiempo determinado guardando los datos digitalizados en los respectivos registros, y sobrescribiéndolos en cada conversión.
 - ADC_START_NOW
 - ADC_START_ON_EINT0
 - ADC_START_ON_CAP01
 - ADC_START_ON_MAT01
 - ADC_START_ON_MAT03
 - ADC_START_ON_MAT10
 - ADC_START_ON_MAT11

Las conversiones se realizan en 65 ciclos de reloj, por tanto, la frecuencia de trabajo del ADC está dada como:

$$f_{ADC} = 65f_{m(max)} = 13MHz > f_m$$

Módulo ADC: Inicialización

1) Se debe energizar el periférico, a partir del registro PCON:

Power Control for Peripherals register (PCONP - address 0x400F C0C4) bit description

Bit	Symbol	Description	Reset value
12	PCADC	A/D converter (ADC) power/clock control bit. Note: Clear the PDN bit in the AD0CR before clearing this bit, and set this bit before setting PDN.	0

```
LPC_SC->PCON |= (1<<12);
```

2) Se debe habilitar el periférico, a partir del registro AD0CR:

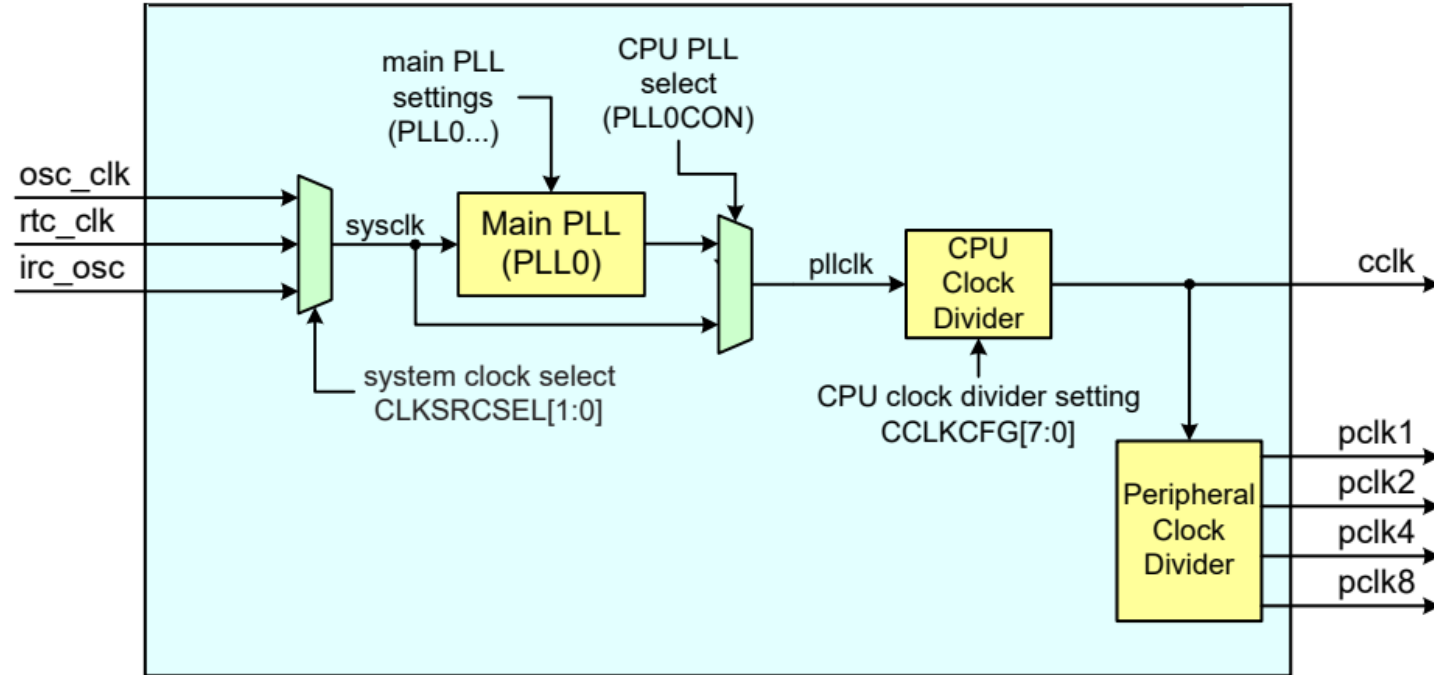
A/D Control Register (AD0CR - address 0x4003 4000) bit description

Bit	Symbol	Value	Description	Reset value
21	PDN	1	The A/D converter is operational.	0
		0	The A/D converter is in power-down mode.	

```
LPC_ADC->ADCR |= (1<<21);
```

Módulo ADC: Inicialización

3) Se debe ajustar la frecuencia del reloj de conversión, a partir del registro AD0CR:



Peripheral Clock Selection register 0 (PCLKSEL0 - address 0x400F C1A8) bit description

Bit	Symbol	Description	Reset value
25:24	PCLK_ADC	Peripheral clock selection for ADC.	00

$$f_{PCLK_{ADC}} = \frac{f_{CCLK}}{PCLK_{ADC}}$$

Módulo ADC: Inicialización

Dado que el ADC solo puede trabajar hasta 13 MHz, se debe calcular el divisor, dado por el registro CLKDIV, para que la frecuencia de muestreo quede justo por debajo o igual a 13 MHz (así la conversión es lo más rápida posible).

A/D Control Register (AD0CR - address 0x4003 4000) bit description

Bit	Symbol	Value	Description	Reset value
15:8	CLKDIV		The APB clock (PCLK_ADC0) is divided by (this value plus one) to produce the clock for the A/D converter, which should be less than or equal to 13 MHz. Typically, software should program the smallest value in this field that yields a clock of 13 MHz or slightly less, but in certain cases (such as a high-impedance analog source) a slower clock may be desirable.	0

$$f_{ADC} = \frac{f_{PCLK_{ADC}}}{(CLKDIV + 1)}$$

$$f_{ADC} = \frac{f_{CCLK}}{PCLKADC(CLKDIV + 1)} = \frac{100[MHz]}{8(0 + 1)} = 12,5[MHz] < 13[MHz]$$

```
LPC_SC->PCLKSEL0 |= (3<<24);  
LPC_ADC->ADCR &= ~(255<<8);
```

Módulo ADC: Inicialización

Nota:

- *El ADC necesita un tiempo mínimo de muestreo para que el capacitor interno de muestreo se cargue al nivel de la señal.*
- *Si la fuente que alimenta el pin del ADC tiene alta impedancia (ej. un divisor resistivo muy grande), la carga es más lenta.*
- *En ese caso, se recomienda bajar la frecuencia del reloj del ADC (poner un CLKDIV más grande), para darle más tiempo al muestreo.*

Módulo ADC: Configuración

4) Se debe seleccionar el modo de disparo a utilizar, a partir del registro AD0CR. En este caso se utilizará el modo de disparo controlado:

A/D Control Register (AD0CR - address 0x4003 4000) bit description

Bit	Symbol	Value	Description	Reset value
16	BURST	1	The AD converter does repeated conversions at up to 200 kHz, scanning (if necessary) through the pins selected by bits set to ones in the SEL field. The first conversion after the start corresponds to the least-significant 1 in the SEL field, then higher numbered 1-bits (pins) if applicable. Repeated conversions can be terminated by clearing this bit, but the conversion that's in progress when this bit is cleared will be completed. Remark: START bits must be 000 when BURST = 1 or conversions will not start. If BURST is set to 1, the ADGINTEN bit in the AD0INTEN register (Table 534) must be set to 0.	0
		0	Conversions are software controlled and require 65 clocks.	

```
LPC_ADC->ADCR &= ~(1<<16);
```


Módulo ADC: Configuración

5) Se debe seleccionar el evento que disparará la conversión del ADC, a partir del registro AD0CR. En este caso se selecciona el inicio debido a MAT0.1:

A/D Control Register (AD0CR - address 0x4003 4000) bit description

Bit	Symbol	Value	Description	Reset value
26:24	START		When the BURST bit is 0, these bits control whether and when an A/D conversion is started:	0
		000	No start (this value should be used when clearing PDN to 0).	
		001	Start conversion now.	
		010	Start conversion when the edge selected by bit 27 occurs on the P2.10 / EINT0 / NMI pin. Set the pin function to EINT0 in PINSEL4 register.	
		011	Start conversion when the edge selected by bit 27 occurs on the P1.27 / CLKOUT / USB_OVRCRn / CAP0.1 pin. Set the pin function to CAP0.1 in PINSEL3 register.	
		100	Start conversion when the edge selected by bit 27 occurs on MAT0.1. Note that this does not require that the MAT0.1 function appear on a device pin.	
		101	Start conversion when the edge selected by bit 27 occurs on MAT0.3. Note that it is not possible to cause the MAT0.3 function to appear on a device pin.	
		110	Start conversion when the edge selected by bit 27 occurs on MAT1.0. Note that this does not require that the MAT1.0 function appear on a device pin.	
		111	Start conversion when the edge selected by bit 27 occurs on MAT1.1. Note that this does not require that the MAT1.1 function appear on a device pin.	

```
LPC_ADC->ADCR &= ~(7<<24);
LPC_ADC->ADCR |= (4<<24);
```

Módulo ADC: Configuración

6) Se debe seleccionar el tipo de flanco de la señal que dará inicio a la conversión, a partir del registro AD0CR. En este caso se selecciona por flanco descendente debido a MAT0.1:

A/D Control Register (AD0CR - address 0x4003 4000) bit description

Bit	Symbol	Value	Description	Reset value
27	EDGE		This bit is significant only when the START field contains 010-111. In these cases:	0
		1	Start conversion on a falling edge on the selected CAP/MAT signal.	
		0	Start conversion on a rising edge on the selected CAP/MAT signal.	

```
LPC_ADC->ADCR |= (1<<27);
```

7) Se debe seleccionar el canal del ADC a ser muestreado, a partir del registro AD0CR:

A/D Control Register (AD0CR - address 0x4003 4000) bit description

Bit	Symbol	Value	Description	Reset value
7:0	SEL		Selects which of the AD0.7:0 pins is (are) to be sampled and converted. For AD0, bit 0 selects Pin AD0.0, and bit 7 selects pin AD0.7. In software-controlled mode, only one of these bits should be 1. In hardware scan mode, any value containing 1 to 8 ones is allowed. All zeroes is equivalent to 0x01.	0x01

```
LPC_ADC->ADCR |= (1<<0);
```

Módulo ADC: Configuración

8) Se debe seleccionar si se generará un pedido de interrupción luego de completada una conversión, a partir del registro AD0INTEN:

A/D Interrupt Enable register (AD0INTEN - address 0x4003 400C) bit description

Bit	Symbol	Value	Description	Reset value
0	ADINTEN0	0	Completion of a conversion on ADC channel 0 will not generate an interrupt.	0
		1	Completion of a conversion on ADC channel 0 will generate an interrupt.	

```
LPC_ADC->ADINTEN |= (1<<0);
```

Nota:

- En este caso también se debe habilitar la interrupción por NVIC:*

```
NVIC_EnableIRQ(ADC_IRQn);
```

Módulo ADC: Verificación

9) Para saber si la conversión de un canal se ha completado, se debe leer el bit DONE del registro AD0DR0:

A/D Data Registers (AD0DR0 to AD0DR7 - 0x4003 4010 to 0x4003 402C) bit description

Bit	Symbol	Description	Reset value
31	DONE	This bit is set to 1 when an A/D conversion completes. It is cleared when this register is read.	NA

```
while(!(LPC_ADC->ADDR0 & (1<<31))){}
```

10) El resultado de la conversión se encuentra alojado en los bits <15:4> del registro AD0DRx :

A/D Data Registers (AD0DR0 to AD0DR7 - 0x4003 4010 to 0x4003 402C) bit description

Bit	Symbol	Description	Reset value
15:4	RESULT	When DONE is 1, this field contains a binary fraction representing the voltage on the AD0[n] pin, as it falls within the range of V_{REFP} to V_{REFN} . Zero in the field indicates that the voltage on the input pin was less than, equal to, or close to that on V_{REFN} , while 0xFFF indicates that the voltage on the input was close to, equal to, or greater than that on V_{REFP} .	NA

```
uint16_t ADC0Value = (LPC_ADC->ADDR0)>>4 & 0xFFF;  
12 bits
```

Ejemplo 1: Interrupción por ADC vía MATCH del TMR0

```
1  /*
2   * @brief   Adquirir una señal analógica de 16kHz por el pin P0.23 cada 1s.
3   * @note    Utilizar la funcionalidad de MATCH del pin P1.29 como base de
4   *          tiempo para la adquisición.
5   */
6
7  /* Directivas de Inclusión */
8  #include "LPC17xx.h"
9  #include "lpc17xx_gpio.h"
10 #include "lpc17xx_pinsel.h"
11 #include "lpc17xx_timer.h"
12 #include "lpc17xx_adc.h"
13
14 /* Definición de Constantes y Macros */
15
16 /* Definición de Variables Globales */
17
18 /* Prototipos de Funciones */
19 void cfgGPIO(void);
20 void cfgTimer(void);
21 void cfgADC(void);
22
23 /* Función Principal */
24 int main(void) {
25
26     cfgGPIO();
27     cfgTimer();
28     cfgADC();
29
30     while(1) {}
31     return 0;
32 }
```

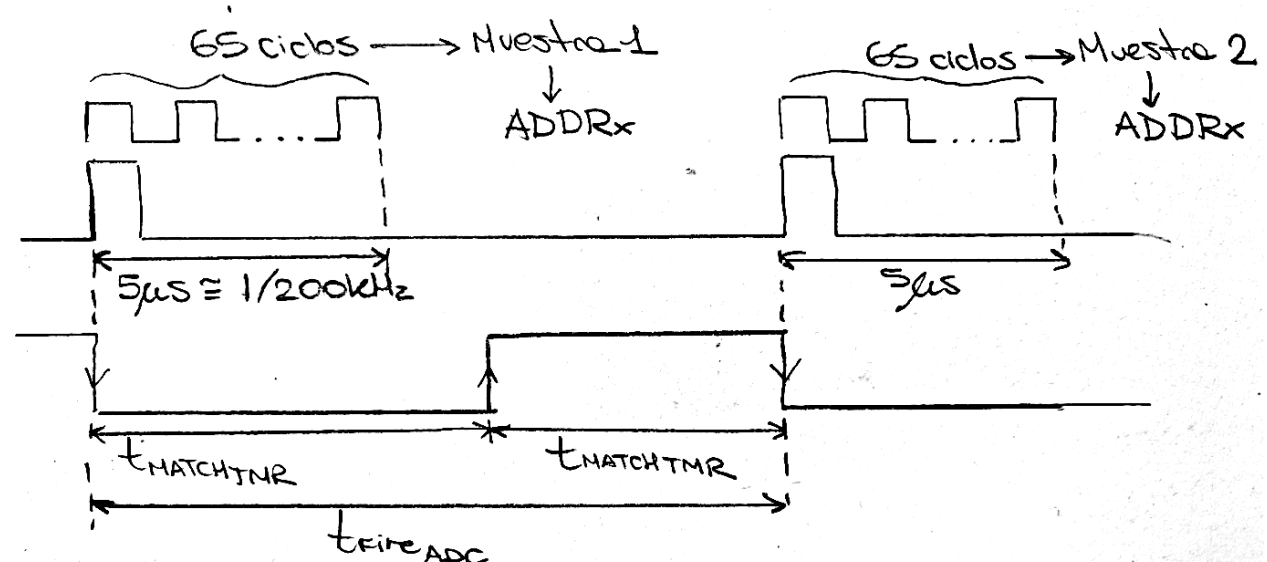
Ejemplo 1: Interrupción por ADC vía MATCH del TMR0

```
34  /* Definiciones de Funciones */
35  void cfgGPIO(void)
36  {
37      PINSEL_CFG_Type cfgPinADC0_CH0;           //Configuración del PIN P0.23
38      PINSEL_CFG_Type cfgPinMAT0_CH1;           //Configuración del PIN P1.29
39
40      cfgPinADC0_CH0.Portnum    = PINSEL_PORT_0; //Selección de Puerto 0
41      cfgPinADC0_CH0.Pinnum     = PINSEL_PIN_23; //Selección de Pin 23
42      cfgPinADC0_CH0.Funcnum    = PINSEL_FUNC_1; //Funcionalidad de P0.23 como AD0.0 (Canal 0)
43      cfgPinADC0_CH0.Pinmode    = PINSEL_PINMODE_TRISTATE; //P0.23 Flotante
44      cfgPinADC0_CH0.OpenDrain  = PINSEL_PINMODE_NORMAL;  //Modalidad sin Drenador Abierto
45
46      cfgPinMAT0_CH1.Portnum    = PINSEL_PORT_1; //Selección de Puerto 1
47      cfgPinMAT0_CH1.Pinnum     = PINSEL_PIN_29; //Selección de Pin 29
48      cfgPinMAT0_CH1.Funcnum    = PINSEL_FUNC_3; //Funcionalidad de P1.29 como MAT0.1
49      cfgPinMAT0_CH1.OpenDrain  = PINSEL_PINMODE_NORMAL; //Modalidad sin Drenador Abierto
50
51      PINSEL_ConfigPin(&cfgPinADC0_CH0);         //Inicialización ADC: P0.23 (AD0.0)
52      PINSEL_ConfigPin(&cfgPinMAT0_CH1);         //Inicialización TMR0: P1.29 (MAT0.1)
53      return;
54  }
55
```

Ejemplo 1: Interrupción por ADC vía MATCH del TMR0

```
56 void cfgTimer(void)
57 {
58     TIM_TIMERCFG_Type    cfgTimerMode;           //Configuración de Timer en Modo Temporizador
59     TIM_MATCHCFG_Type    cfgTimerMatch;          //Configuración del Tiempo de Desborde de MATCH
60
61     cfgTimerMode.PrescaleOption = TIM_PRESCALE_USVAL; //Base de Tiempo del Prescaler a partir de valor en µs
62     cfgTimerMode.PrescaleValue  = 1000;           //Base de Tiempo de Desborde de Prescaler [µs] (Time_Overflow_PC)
63
64     cfgTimerMatch.MatchChannel  = 1;              //Habilitación de Canal de MATCH0: (MR1)
65     cfgTimerMatch.MatchValue    = 499;           //Valor Máximo para TC (MR)
66     cfgTimerMatch.IntOnMatch    = DISABLE;        //Habilitación de Interrupción por MATCH: (MR1I=0)
67     cfgTimerMatch.ResetOnMatch  = ENABLE;         //Habilitación de Reset por MATCH: (MR1R=1)
68     cfgTimerMatch.StopOnMatch   = DISABLE;        //Deshabilitación de Stop por MATCH: (MR1S=0)
69     cfgTimerMatch.ExtMatchOutputType = TIM_EXTMATCH_TOGGLE; //Modalidad TOGGLE en P1.29
70
71     TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTimerMode); //Configuración del Modalidad del TIMER0
72     TIM_ConfigMatch(LPC_TIM0, &cfgTimerMatch);          //Configuración del TIMER0 como Temporizador
73     TIM_Cmd(LPC_TIM0, ENABLE);                        //Habilitación del TIMER0
74
75     return;
76 }
77
```

$$\begin{aligned} t_{\text{delay}} &= \frac{\text{toggle}}{2} t_{\text{MATCH}_{\text{TMR0}}} \Rightarrow 1\text{s} \\ MR &= \frac{t_{\text{MATCH}_{\text{TMR0}}}}{t_{\text{OVERFLOW}_{\text{PC}}}} - 1 \\ &= \frac{\frac{t_{\text{delay}}}{2}}{t_{\text{OVERFLOW}_{\text{PC}}}} - 1 \\ &= \frac{\frac{1\text{s}}{2}}{1000\mu\text{s}} - 1 \\ &= 499 \end{aligned}$$



Ejemplo 1: Interrupción por ADC vía MATCH del TMR0

```
78 void cfgADC(void){
79     ADC_Init(LPC_ADC, 32000)           //Iniciación del ADC con Frecuencia de Muestreo f_m[Hz]
80     //LPC_SC->PCON |= (1<<12);         //Encendido del Periférico (AD0CR ON) - [Default: OFF]
81     //LPC_ADC->ADCR |= (1<<21);         //Habilitación del Periférico (AD0CR ON): (PDN=1)
82     //LPC_SC->PCLKSEL0 |= (3<<24);      //PCLK_ADC0 = CCLK/8 ⇒ f_PCLK = f_CCLK/8 = 12,5MHz
83     //LPC_ADC->ADCR &= ~(255<<8);       //CLKDIV = (PCLK_ADC0/ADCCLK) - 1: (CLKDIV=256) ⇒ f_ADC = f_PCLK/1 = 12,5MHz
84     ADC_BurstCmd(LPC_ADC, DISABLE);    //Deshabilitación del Modo Burst para ADC: (BURST=0)
85     //LPC_ADC->ADCR &= ~(1<<16);        //Deshabilitación del Modo Burst para ADC: (BURST=0)
86     ADC_StartCmd(LPC_ADC, ADC_START_ON_MAT01); //Iniciación de Conversión debido a MAT0.1: (START=100)
87     //LPC_ADC->ADCR &= ~(7<<24);       //Mascara para START: (START=000)
88     //LPC_ADC->ADCR |= (4<<24);         //Iniciación de Conversión debido a MAT0.1: (START=100)
89     ADC_EdgeStartConfig(LPC_ADC, ADC_START_ON_FALLING); /* Conversión ante Flanco Descendente
90                                                         * debido a MAT0.1: (EDGE=1)
91                                                         */
92     //LPC_ADC->ADCR |= (1<<27);         //Iniciación de Conversión ante Flanco Descendente debido a MAT0.1: (EDGE=1)
93     ADC_ChannelCmd(LPC_ADC, ADC_CHANNEL_0, ENABLE); //Funcionalidad de P0.23 como AD0.0 (Canal 0)
94     //LPC_ADC->ADCR |= (1<<0);          //Funcionalidad de P0.23 como AD0.0 (Canal 0)
95     ADC_IntConfig(LPC_ADC, ADC_ADINTEN0, ENABLE);   /* Habilitación de Interrupción ante Finalización de Conversión
96                                                         * por AD0.0
97                                                         */
98     //LPC_ADC->ADINTEN |= (1<<0);       //Habilitación de Interrupción ante Finalización de Conversión por AD0.0
99     NVIC_EnableIRQ(ADC_IRQn);          //Habilitación de Interrupción por ADC0
100    return;
101 }
102
103 void ADC_IRQHandler(void){
104     while(!(ADC_ChannelGetStatus(LPC_ADC, ADC_CHANNEL_0, ADC_DATA_DONE))){}; //Espera Fin de Conversión de AD0.0
105     //while(!(LPC_ADC->ADDR0 & (1<<31))){} //DONE_CH0 = 1?
106
107     uint16_t ADC0Value = ADC_ChannelGetData(LPC_ADC, ADC_CHANNEL_0); //Guarda Valor Digitalizado de AD0.0
108     //LPC_ADC->ADDR0 &= (1<<31); //Limpia DONE del CH0
109     //uint16_t ADC0Value = (LPC_ADC->ADDR0)>>4 & 0xFFFF; //Guarda Valor Digitalizado de AD0.0
110
111     return;
112 }
113
```


Ejercicio 1: Interrupción por ADC

En base al ejemplo 1, modificar la configuración del ADC para considerar la utilización de dos canales de conversión. Determinar a que frecuencia se encuentra muestreando cada canal.

Ejercicio 2: Interrupción por ADC

Configurar 4 canales del ADC para que funcionando en modo burst se obtenga una frecuencia de muestreo en cada uno de 50Kmuestras/seg. Suponer un CCLK = 100 MHz.

Ejercicio 3: Interrupción por ADC

En base al ejemplo 1, modificar el valor del Match para que la frecuencia de muestreo del ADC sea de 20 Kmuestras/seg. El resultado de la conversión debe mostrarse por 12 pines de salida del puerto GPIO0.